Laboration 3 – Binärt sökträd

Viktigt! *Som vanligt, läs "Information och regler kring laborationerna".*

Din lösning ska lämnas in i Canvas, gör inlämningn så snart du känner dig färdig med den. Du ska också lämna in en screenshot där det framgår att testfunktionen har körts utan att några asserts aktiverats.

Läs varje deluppgift noggrant då det är viktigt att du följer instruktionerna för att du ska få ihop allt rätt. Om du undrar över något eller tycker att något är otydligt, hör av dig och fråga.

Vid inmatning från användaren kan du anta att korrekt typ matas in.

Se till att vara förberedd inför labbtillfällena, så att du verkligen hinner få hjälp med det du eventuellt har problem med.

Tänk igen om hur du ska ta dig an uppgiften och planera lösningen innan du börjar koda – vad är funktionens uppgift och vad måste hända rent logiskt. Rita gärna upp olika scenarion för att förstå vilka problem som kan uppstå.

Uppgift 3.1: Ladda ner och kompilera labbskelettet

Hämta filerna i den zippade filen BSTree.zip som finns i Modul C i Canvas. Öppna sedan ett nytt projekt i din utvecklingsmiljö. Lägg till de tre filerna i projektet. Kontrollera att det går att kompilera, du kommer troligtvis få varningar att tree samt de statiska funktionerna inte används, så ska det vara.

Uppgift 3.2: Binärt sökträd & menyprogram

Du ska nu implementera alla funktioner som finns deklarerade i `BSTree.h`. Det finns också några statiska tomma funktioner i `BSTree.c`. Du kan implementera dessa i den ordning du tycker är lättast. Tänk också på att många av funktionerna kommer att använda varandra. Ett förslag är att du börjar med den statiska funktionen `createNode()` i `BSTree.c`. Denna funktion skapar och allokerar plats för trädnoder och behövs för att kunna sätta in noder i trädet.

I `test_BSTree.c` finns en menyfunktion du kan använda, det är givetvis helt ok att istället skriva en egen menyfunktion.

Eftersom det är många funktioner som ska implementeras så bör du skriva en funktion i taget och testa den ordentligt via menyn för att se så att den funkar som den ska innan du går vidare till nästa funktion. Detta kommer att spara dig tid i Uppgift 3.3.

Funktionerna `writeSortedToArray()` och `buildTreeSortedFromArray()` anv ands av balanseringsfunktionen och behöver bara användas ifall du väljer att implementera balanseringen på det sätt som föreslås i kommentarerna för `balanceTree()`.

Uppgift 3.3: Testa trädet

När samtliga funktioner är implementerade ska du testa ditt träd. Kör testfunktionen i `test_BSTree.c`, om en assert aktiveras behöver du ta reda på vilka fel som finns i din implementation (kommentarerna i funktionen bör hjälpa).

OBS! Traverseringsfunktionerna Preorder, Inorder och Postordet måste testas manuellt via menyn.